# Infra

Morning Flower and Evening Oath

Python、C++ 进阶、Shell 与赛场调试

假定 Linux、Bash、GCC 13+ 与 C++23、Python 3.11+。

# 目录


**第一章 Python** **1**

1.1 精确整数、分数与小数 . . . 1
1.2 枚举、容器与记忆化 . . . 2
1.3 随机造数与打表找规律 . . . 3
1.4 输入输出与容易踩的坑 . . . 4

**第二章 C++ 进阶** **5**

2.1 优化选项与局部 pragma . . . 5
2.2 浮点优化会改变答案 . . . 6
2.3 O2 基线下怎样测 . . . 6
2.4 先减少复制、分配和跨步访问 . . . 7
2.5 复用缓冲与输入输出 . . . 8
2.6 整数输入输出：EOF 与极值 . . . 9
2.7 逐行解析整数 . . . 11
2.8 先检查自动向量化 . . . 12
2.9 AVX2 类型与常用指令 . . . 12
2.10 AVX2 批量加法与条件计数 . . . 13
2.11 重排与 AVX-512 掩码尾部 . . . 14
2.12 原生 Linux 实测 . . . 14
2.13 编译警告与检查配置 . . . 15
2.14 读懂第一条错误 . . . 16
2.15 值、引用与调用边界 . . . 17
2.16 std::move：允许移动，不是立即移动 . . . 17
2.17 vector、array、deque 与字符串 . . . 17
2.18 set、map 与哈希表 . . . 17
2.19 queue、stack 与优先队列 . . . 18
2.20 排序、查找与区间算法 . . . 19
2.21 pair、tuple、optional 与回调 . . . 19
2.22 数值累加与前缀和 . . . 20
2.23 complex 与 FFT . . . 20
2.24 C++20 位操作与零边界 . . . 20
2.25 计时与卡时 . . . 21
2.26 随机引擎、分布与抽样 . . . 21
2.27 对拍用造数据函数 . . . 22
2.28 生成器入口 . . . 23
2.29 整数键防卡哈希 . . . 24
2.30 GNU 扩展与查询范围 . . . 24
2.31 Boost 任意精度整数 . . . 24
2.32 __int128 的十进制读写 . . . 25
2.33 strict 与 GNU 模式的差异 . . . 26
2.34 GNU 位函数与 bitset 遍历 . . . 27

**第三章 Shell 脚本与常用命令** **28**

3.1 编译并运行 . . . 28
3.2 对拍与保存反例 . . . 29
3.3 完整造数与暴力示例 . . . 30

**第四章 赛场环境与调试** **31**

4.1 开赛与热身检查清单 . . . 31
4.2 短探针与速度校验 . . . 32
4.3 提交前核对 . . . 32
4.4 GDB、assert 与 LOCAL . . . 33
4.5 测时间、峰值内存与限制 . . . 34
4.6 交互题：刷新、协议与失败退出 . . . 35
4.7 最短本地裁判 . . . 36

4.8 分册跳转索引 . . . . . 37

# 第一章　Python

## 1.1　精确整数、分数与小数

需要整数答案时保留整数运算；不要先除成 float 再转回 int。

| 用途 | 最短写法与坑点 |
|---|---|
| 整数平方根 | `isqrt(n)`，要求非负；返回向下取整的精确结果。 |
| 组合数、最大公约数 | `comb(n,k)`、`gcd(a,b)`、`lcm(a,b)`。 |
| 模幂、模逆 | `pow(a,e,m)`；逆元用 `pow(a,-1,m)`，不互质抛 ValueError。 |
| 整数除法 | `//` 向下取整，C++ 向零截断；Python 余数与除数同号或为零。 |
| 有理数 | `Fraction(a,b)` 自动约分；十进制输入用字符串，避免先变 float。 |
| 高精度小数 | `Decimal("0.1")`；context 的 prec 是有效数字数，并非小数位数。 |

```
from math import comb, gcd, isqrt, lcm
from fractions import Fraction
from decimal import Decimal, localcontext

n = 10**100
assert isqrt(n - 1) == 10**50 - 1
assert comb(100, 50) == 100891344545564193334812497256
assert gcd(12, 18) == 6
assert lcm(12, 18) == 36
assert pow(3, 100, 7) == 4
assert pow(3, -1, 7) == 5
assert divmod(-7, 3) == (-3, 2)
assert (-7) // 3 == -3
assert -(7 // 3) == -2

a = Fraction(1, 3) + Fraction(1, 6)
assert (a.numerator, a.denominator) == (1, 2)
assert Fraction("0.1") == Fraction(1, 10)
assert Fraction(0.1) != Fraction(1, 10)
with localcontext() as ctx:
    ctx.prec = 60
    x = Decimal(2).sqrt()
    assert abs(x * x - 2) < Decimal("1e-58")
    assert Decimal("0.1") * 3 == Decimal("0.3")
print("numbers: pass")
```

`Fraction` 的分子分母可能快速膨胀；`Decimal` 仍有舍入，不能当作任意精确实数。

## 1.2 枚举、容器与记忆化

`combinations` 枚举选下标的组合，`permutations` 枚举排列，`product` 枚举笛卡尔积。输入有重复值时不自动去重。它们产生迭代器；大规模枚举直接遍历，别先转成 list。

```
from collections import Counter, defaultdict, deque
from itertools import accumulate, combinations, permutations, product
from heapq import heapify, heappop, heappush
from bisect import bisect_left, bisect_right
from functools import cache

assert list(combinations(range(4), 2))[-1] == (2, 3)
assert len(list(permutations(range(4)))) == 24
assert len(list(product(range(2), repeat=5))) == 32
assert list(accumulate([2, -1, 4], initial=0)) == [0, 2, 1, 5]
assert Counter("ababa")["a"] == 3
g = defaultdict(list)
g[2].append(3)
assert g[2] == [3]
q = deque([1, 2])
q.appendleft(0)
assert q.popleft() == 0
h = [5, 2, 7]
heapify(h)
heappush(h, 1)
assert heappop(h) == 1
a = [1, 2, 2, 4]
assert bisect_right(a, 2) - bisect_left(a, 2) == 2

@cache
def paths(r, c):
    if r == 0 or c == 0:
        return 1
    return paths(r - 1, c) + paths(r, c - 1)

assert paths(10, 10) == 184756
paths.cache_clear()
rows = [[0] * 2 for _ in range(3)]
rows[0][0] = 7
assert rows[1][0] == 0
print("containers: pass")
```

heapq 默认小根堆；大根堆可存负值。bisect 要求序列有序，二分 O(log n)，向 list 中间插入仍 O(n)。Counter 读不存在的键返回 0；defaultdict 下标读取可能插入新键。cache 参数必须可哈希，多组改变外部状态后清缓存。递归深度仍受限。

## 1.3 随机造数与打表找规律

固定种子复现失败输入；不同 Python 版本不承诺所有抽样函数逐项一致，保存实际输入。randint 两端包含；randrange 右端不含，第三参数是步长。sample 无放回抽取位置，输入有重复值时结果仍可能重复；shuffle 原地改序列并返回 None。

```
import random

rng = random.Random(2026)
a = rng.randint(-5, 5)
b = rng.randrange(2, 10, 2)
c = rng.choice("abc")
v = rng.sample(range(100), 10)
rng.shuffle(v)
assert -5 <= a <= 5
assert b in (2, 4, 6, 8)
assert c in "abc"
assert len(set(v)) == 10
rng.seed(42)
first = [rng.randrange(1000) for _ in range(10)]
rng.seed(42)
assert first == [rng.randrange(1000) for _ in range(10)]
print("random: pass")
```

下例暴力统计无相邻 1 的二进制串，检查前 11 项是否符合猜测。有限打表只能发现规律；本例递推证明为按末位 0、末尾 01 分类，另处理长度 0 和 1。

```
from itertools import product

# Binary strings with no adjacent ones, including the empty string.
counts = []
for n in range(11):
    good = 0
    for a in product(range(2), repeat=n):
        if all(a[i - 1] + a[i] < 2 for i in range(1, n)):
            good += 1
    counts.append(good)
    print(n, good)
assert counts[:2] == [1, 2]
assert all(counts[n] == counts[n - 1] + counts[n - 2]
           for n in range(2, len(counts)))
```

## 1.4 输入输出与容易踩的坑

```python
import sys

# Input: n followed by n integers, possibly across multiple lines.
it = iter(map(int, sys.stdin.buffer.read().split()))
n = next(it)
a = [next(it) for _ in range(n)]
sys.stdout.write(str(sum(a)) + "\n")
sys.stdout.write(" ".join(map(str, a)) + "\n")
```

`read().split()` 同时保存全部 token，大输入可改成逐行读。bytes 下标得到整数；读文本才得到单字符字符串。大量输出用 join 或分块缓冲，避免循环逐字符拼接字符串。

| 场景 | 做法 |
|---|---|
| 二维数组 | `[[0]*m for _ in range(n)]`；`[[0]*m]*n` 共享同一行。 |
| 复制 | `b=a` 为别名；`a[:]` 只复制外层。 |
| 递归 | `sys.setrecursionlimit(k)` 调整解释器限制，不增加操作系统栈与内存。深递归须在比赛环境实测。 |
| 选择解释器 | CPython 启动快、C 实现内置操作强；PyPy 的 JIT 可能加速长时间纯 Python 循环，但有预热和额外内存。用相同最大数据分别实测。 |
| 大整数 | int 不溢出，但操作成本随位数增长；避免 `int(a/b)` 做大整数除法。 |
| 十进制转换 | CPython 3.11+ 可限制十进制转换位数；只对可信比赛输入按下例关闭。 |

```python
import sys

# Trusted contest input only; affects conversions, not int arithmetic.
sys.set_int_max_str_digits(0)
x = int("9" * 5000)
assert len(str(x)) == 5000
assert len(str(x + 1)) == 5001
print("digits: pass")
```

该限制作用于十进制等转换，不限制 int 本身的算术位数；二进制、十六进制转换不受同一限制。

# 第二章　C++ 进阶

## 2.1　优化选项与局部 pragma

先选合适算法和数据结构，再定位热点；以评测的 O2 配置为基线实测。优化不是纠正未定义行为的工具，调试版与提交版要分别运行。

| 选项 | 用法与代价 |
|---|---|
| `-O2` | 常用优化基线。具体启用项依 GCC 版本和目标而变。 |
| `-O3` | 增加循环等变换，可能增大代码；不保证比 O2 快。 |
| `-funroll-loops` | 尝试展开循环，减少部分循环控制开销；代码膨胀可能伤害指令缓存。 |
| `-Ofast` | 含 fast-math 等放宽语义的优化；浮点重排、NaN/Inf、带符号零等处理可能变化。 |
| `GCC optimize` | 给后续函数定义设置优化选项，不追溯先前定义。 |
| `GCC target` | 为后续函数选目标特性；启用指令集不等于检测运行 CPU。 |

下面仅在一个函数上试 O3 和循环展开；push/pop 恢复进入前的设置。

```cpp
#include <bits/stdc++.h>
using namespace std;

#pragma GCC push_options
#pragma GCC optimize("O3", "unroll-loops")
uint64_t sum_values(const vector<unsigned>& a) {
    uint64_t sum = 0;
    for (unsigned x : a) {
        sum += x;
    }
    return sum;
}
#pragma GCC pop_options

int main() {
    vector<unsigned> a{1, 2, 3};
    cout << sum_values(a) << '\n';
}
```

把 pragma 放在 `#include` 前，头文件内后续定义的函数、内联函数和模板也可能受影响；放在 include 后不追溯已读入的定义。头文件自己的属性或 push/pop 还可能调整作用范围。优先在头文件之后，只包住已测过的热点；不要无差别堆叠全局“火车头”。`reset_options` 恢复命令行的默认选项，`pop_options` 恢复上次 push 的设置。

x86 的 `target("avx2")` 允许生成 AVX2 代码，不自动判断评测 CPU 是否支持。执行 CPU 或操作系统未支持的指令可能触发非法指令退出（常见 SIGILL）。更高指令集不必然更快；目标函数与调用者特性不一致也可能导致内联编译错误。热身时确认评测 CPU、系统和编译器支持，再选择目标指令集。

## 2.2 浮点优化会改变答案

下例输入 `10000000000000000 1`。普通 double 计算先把加 1 舍入掉；允许代数重排后，编译器可以把整个表达式化为 y。

```cpp
#include <bits/stdc++.h>
using namespace std;

__attribute__((noinline))
double expression(double x, double y) {
    return (x + y) - x;
}

int main() {
    double x;
    double y;
    cin >> x >> y;
    cout << setprecision(17) << expression(x, y) << '\n';
}
```

本机 GNU GCC16.2.0 实测：O2、O3 输出 0，Ofast 输出 1。这足以说明 Ofast 并非单纯 “更努力优化”；几何谓词和浮点误差分析不能默认沿用原来的语义。

```
g++ fast_math.cpp -std=c++23 -O2 -o math_o2
g++ fast_math.cpp -std=c++23 -Ofast -o math_fast
printf '10000000000000000 1\n' | ./math_o2
printf '10000000000000000 1\n' | ./math_fast
```

pragma 的 include 边界也已用同一表达式实测：头文件定义函数 H，include 后定义函数 L，pop 后定义函数 R，三个函数都保留独立调用。外层按 O2 编译。

| Ofast pragma 的位置 | H / L / R 输出 |
|---|---|
| 放在 include 前，L 之后 pop | 1 / 1 / 0 |
| 放在 include 后，L 之后 pop | 0 / 1 / 0 |

这说明局部选项可以恢复，也说明 include 前的 pragma 会影响头文件定义。不要因为 main 在 pop 之后，就认为它调用的所有函数也恢复成了 O2。

## 2.3 O2 基线下怎样测

相同输入、校验答案、交错运行候选、多次取中位数；预先约定是否包含初始化、分配和析构。计时区间内不能打印调试信息。必须使用结果，防止编译器删除无用计算。微基准只定位成本，最终仍应跑原题最大数据。

后页实测环境为 Apple M5、macOS arm64、GNU GCC16.2.0、C++23 与 O2，每项 9 个新进程，表中单位为毫秒。计时使用 steady_clock；每次校验整数和。算法主体读取 $2048^2$ 个 int，值为 `i%1009`。同一按行求和循环在 O2、O3、O2 加 unroll 下的中位数见表；差异很小，没有测出稳定的大幅收益。

| 编译选项 | 按行求和 (ms) |
|---|---:|
| O2 | 1.056 |
| O3 | 1.069 |
| O2 + funroll-loops | 1.094 |

## 2.4 先减少复制、分配和跨步访问

| 对比 | 前者 (ms) | 后者 (ms) |
|---|---|---|
| 逐个扩容 / 一次 reserve | 5.152 | 3.998 |
| 按值复制 / const 引用 | 2.245 | 1.030 |
| 按值复制 / 转移所有权 | 2.245 | 1.081 |
| 按列访问 / 按行访问 | 7.308 | 1.048 |
| 每轮新分配 / clear 复用 | 5.147 | 4.210 |
| 默认 cin / 关闭同步和绑定 | 307.987 | 15.015 |

扩容组从空 vector 开始，计入 reserve、push 和求和；其他数组组先在计时外填好数据。传值组计入参数构造、求和和参数析构；move 后原数组不再使用，引用组仍借用原数组。这些调用的所有权要求不同，先满足接口语义，再考虑快慢。

| 热点 | 可直接采用的改动 |
|---|---|
| vector 逐个追加 | 已知数量时先 reserve(n)，避免多次搬移。别每次追加都 reserve(size()+1)，这会破坏几何扩容。 |
| 只读大对象 | 参数用 const 引用；遍历大元素用 `const auto&`，避免无意复制。 |
| 转交所有权 | 接口按值接收且原对象之后不用时，传 std::move；const 对象通常仍复制。 |
| 多轮临时数组 | 尺寸相近时复用缓冲区，clear 保留 capacity；峰值很大且不再需要时再考虑释放。 |
| 连续矩阵 | 行主序存储通常让列下标在内层，尽量连续访问；不要在内层反复判断遍历模式。 |
| 结构体数组 | 先看 sizeof 与填充，热点只读少数字段时再试分离字段数组，按实际负载测。 |

reserve 只增加容量，不产生可下标访问的元素；过度预留会占内存。少量标量按值传递通常更简单，不能把所有参数都改为引用。move 传给 const 引用不会转交所有权，随意给返回值加 move 也可能妨碍复制消除。`inline` 不强制机器码内联，`register` 不是通用提速手段。

本次按行/按列都累加相同元素；行主序下前者内层连续，后者步长为 2048 个 int。修改访问顺序必须保留算法依赖，例如背包方向不能为了局部性随意倒置。

## 2.5 复用缓冲与输入输出

此例比较每轮释放重建与 clear 复用，2048 轮、每轮 4096 个 int，累计校验所有元素。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(int argc, char** argv) {
    bool fresh = argc > 1 && string(argv[1]) == "fresh";
    vector<int> a;
    uint64_t sum = 0;
    auto start = chrono::steady_clock::now();
    for (int round = 0; round < 2048; round++) {
        if (fresh) {
            vector<int>().swap(a);
        } else {
            a.clear();
        }
        for (int i = 0; i < 4096; i++) {
            a.push_back(i ^ round);
        }
        sum += accumulate(a.begin(), a.end(), uint64_t(0));
    }
    double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - start).count();
    cout << sum << ' ' << fixed << setprecision(6) << ms << '\n';
}
```

输入组从同一磁盘文件读取 100 万个十进制整数（值仍为 `i%1009`），计时包含解析和求和。关闭同步要在 I/O 前做；之后避免混用 stdio 与 iostream，交互题应显式刷新。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(int argc, char** argv) {
    bool fast = argc > 1 && string(argv[1]) == "fast";
    if (fast) {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);
    }
    auto start = chrono::steady_clock::now();
    uint64_t sum = 0;
    int x;
    while (cin >> x) {
        sum += x;
    }
    double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - start).count();
    cout << sum << ' ' << fixed << setprecision(6) << ms << '\n';
}
```

通常用 `'\n'` 代替逐行 endl；endl 还会刷新。取消 cin 与 cout 绑定后，不能依赖下一次读取帮你刷新提示。

## 2.6 整数输入输出：EOF 与极值

以下读写 signed64 十进制整数，接受单个正负号与前导零。要求普通文本、C locale、long long 为 64 位。getchar 返回 int 以区分所有字节与 EOF；跳过空白时也检查 EOF。关闭 iostream 同步后不要混用这组 stdio 函数。`read_ll(x)` 成功返回 true；EOF、非法或越界时返回 false 且 x 不变。失败后停止解析，不提供恢复到下一词的接口；非法输入与正常 EOF 不区分。合法词后必须是空白或 EOF。负数先按无符号幅值累积，再恢复符号，支持最小负数。`write_ll(x)` 不自动换行，不对最小负数直接取负。每个数字 O(位数)，临时空间 O(位数)；使用 stdio 已有缓冲，不宣称优于所有 OJ 上的 cin/scanf。

```
bool read_ll(long long &x)
{
    int c = getchar();
    while (c != EOF && isspace(c))
        c = getchar();
    if (c == EOF)
        return false;
    bool neg = c == '-';
    if (c == '-' || c == '+')
        c = getchar();
    if (c < '0' || c > '9')
        return false;
    unsigned long long v = 0;
    unsigned long long lim = (1ULL << 63) - !neg;
    while (c >= '0' && c <= '9')
    {
        int d = c - '0';
        if (v > (lim - d) / 10)
            return false;
        v = v * 10 + d;
        c = getchar();
    }
    if (c != EOF && !isspace(c))
        return false;
    x = neg && v ? -static_cast<long long>(v - 1) - 1 : static_cast<long long>(v);
    return true;
}
```

输出函数与完整调用：

```
void write_ll(long long x)
{
    auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
    string s;
    do
    {
        s.push_back('0' + v % 10);
        v /= 10;
    } while (v);
    if (x < 0)
        putchar('-');
    for (int i = int(s.size()) - 1; i >= 0; i--)
        putchar(s[i]);
}
```

```
int main()
{
    long long x;
    while (read_ll(x))
    {
        write_ll(x);
        putchar('\n');
    }
}
```

## 2.7 逐行解析整数

每行是一组 signed64 整数，以空白分隔；空行得到空数组。示例输出数量及规范化的数；整行存在非法词或越界则输出 invalid，不处理已解析前缀。先跳过空白再检查 EOF，避免把末尾越界但同时到 EOF 误认为成功。若前面使用过 cin 提取，再调用 getline 前应按题意处理遗留换行。此处直接 getline，不自动吞掉有意义的空行。

```
int main()
{
    string line;
    while (getline(cin, line))
    {
        istringstream in(line);
        vector<long long> a;
        bool ok = true;
        while (in >> ws && !in.eof())
        {
            long long x;
            if (!(in >> x))
            {
                ok = false;
                break;
            }
            int c = in.peek();
            if (c != char_traits<char>::eof() && !isspace(c))
            {
                ok = false;
                break;
            }
            a.push_back(x);
        }
        if (!ok)
        {
            cout << "invalid\n";
            continue;
        }
        cout << a.size();
        for (long long x : a)
            cout << ' ' << x;
        cout << '\n';
    }
}
```

原稿 gets 没有缓冲区长度；不采用。strtok 会修改字符数组且分隔符是字符集合，传入单个空格不等于所有空白；sscanf 必须检查返回值，转换失败时不能复用旧变量。上述整行方案使用标准流并显式拒绝非完整整数词。

## 2.8 先检查自动向量化

先写连续数组上的简单循环，以 O2 编译并看报告；循环间依赖、指针可能重叠、复杂分支都可能阻止向量化。

```
g++ main.cpp -std=c++23 -O2 -mavx2 \
    -fopt-info-vec-optimized -o main
g++ main.cpp -std=c++23 -O2 -mavx2 \
    -fopt-info-vec-missed -o main
objdump -d -C main > main.asm
```

报告要定位到目标循环；反汇编检查目标函数里的 ymm/zmm 指令，不能只在整份程序里搜到一条就算通过。`__restrict` 承诺指针访问满足不别名要求，只能在实际满足时使用。手写 intrinsics 也需要逐项校验结果，并与编译器生成的版本比较。

## 2.9 AVX2 类型与常用指令

包含 `<immintrin.h>`。命令行用 `-mavx2`，或用下面的 push/target/pop 只包住目标函数。`__m256i` 是 256 位整数向量，可按 8 个 32 位或 4 个 64 位整数操作；`__m256` 为 8 个 float，`__m256d` 为 4 个 double。整数类型本身不区分有符号性，含义由具体指令决定。下表名称均省略前缀 `_mm256_`。

| 后缀 | 用途与边界 |
|---|---|
| `setzero_si256` / `set1_epi32` | 全零 / 广播一个 32 位值。 |
| `loadu_si256` / `storeu_si256` | 读写 32 字节，不要求 32 字节对齐；所有字节仍须有效。 |
| `load_si256` / `store_si256` | 地址必须 32 字节对齐；普通 vector 不保证这一点。 |
| `add_epi32` / `sub_epi32` | 每个 32 位槽独立加减，低 32 位结果。 |
| `mullo_epi32` | 每槽乘积低 32 位；不是 8 个完整 64 位乘积。 |
| `and_si256` / `or_si256` | 逐位与 / 或。 |
| `xor_si256` / `andnot_si256` | 异或 / `( a)&b`，注意 andnot 参数次序。 |
| `cmpeq_epi32` / `cmpgt_epi32` | 相等 / 有符号大于；真槽全 1，假槽全 0。 |
| `movemask_epi8` | 取 32 个字节各自的最高位，组成 32 位掩码。 |
| `castsi256_ps` + `movemask_ps` | 不转换数值地重解释，再取 8 个 32 位槽的最高位。 |
| `permutevar8x32_epi32` | 按 8 个下标重排整个 256 位向量的 32 位槽。 |
| `shuffle_epi8` | 每个 128 位半区内部按字节重排，不能任意跨半区。 |

无符号 32 位比较可先把两边异或最高位 `0x80000000u`，再作有符号比较。`cast` 不执行浮点数值转换；下面的 movemask 只提取比较结果，不进行浮点运算。

## 2.10 AVX2 批量加法与条件计数

加法采用 uint32_t，标量与向量都按模 $2^{32}$ 回绕；不能直接以可能溢出的有符号加法作参考。约定 n 非负，三个数组至少 n 个元素，输出与输入不重叠。每次只读取完整的 8 个元素，余数走标量。循环写 `i<=n-8`，避免 `i+8` 在 n 接近 INT_MAX 时溢出。

```
#include <cstdint>
#include <immintrin.h>

#pragma GCC push_options
#pragma GCC target("avx2")

// n >= 0; a, b, out each contain n uint32_t; output does not overlap input.
void add_avx2(const uint32_t* a, const uint32_t* b, uint32_t* out, int n) {
    int i = 0;
    for (; i <= n - 8; i += 8) {
        auto x = _mm256_loadu_si256((const __m256i*)(a + i));
        auto y = _mm256_loadu_si256((const __m256i*)(b + i));
        auto z = _mm256_add_epi32(x, y);
        _mm256_storeu_si256((__m256i*)(out + i), z);
    }
    for (; i < n; i++) {
        out[i] = a[i] + b[i];
    }
}
```

计数使用有符号 32 位比较；提取每槽一个 bit，再统计 1 的个数。以下函数接在同一个 target 范围中，末尾 pop 恢复原来的目标选项。

```
int count_less_avx2(const int32_t* a, int n, int32_t bound) {
    auto limit = _mm256_set1_epi32(bound);
    int count = 0;
    int i = 0;
    for (; i <= n - 8; i += 8) {
        auto x = _mm256_loadu_si256((const __m256i*)(a + i));
        auto mask = _mm256_cmpgt_epi32(limit, x);
        unsigned bits = _mm256_movemask_ps(_mm256_castsi256_ps(mask));
        count += __builtin_popcount(bits);
    }
    for (; i < n; i++) {
        count += a[i] < bound;
    }
    return count;
}

#pragma GCC pop_options
```

## 2.11 重排与 AVX-512 掩码尾部

逆序 8 个 32 位整数：setr 按书写次序设置槽，槽 0 对应数组第一个元素。输入和输出至少 8 个元素。

```
#pragma GCC push_options
#pragma GCC target("avx2")
void reverse_eight(const int32_t* a, int32_t* out) {
    auto x = _mm256_loadu_si256((const __m256i*)a);
    auto order = _mm256_setr_epi32(7, 6, 5, 4, 3, 2, 1, 0);
    auto y = _mm256_permutevar8x32_epi32(x, order);
    _mm256_storeu_si256((__m256i*)out, y);
}
#pragma GCC pop_options
```

AVX-512 的整数向量为 `__m512i`，可容纳 16 个 32 位整数；`__mmask16` 的 bit j 控制槽 j。下例只需 AVX512F；AVX512BW、DQ、VL 是额外子集，不能由 F 推断它们都可用。最后不足 16 项时，只访问掩码选中的槽，不越过数组尾部。maskz 加载将未选中的槽置零，mask 存储不写未选中的槽。

```
#pragma GCC push_options
#pragma GCC target("avx512f")

void add_avx512(const uint32_t* a, const uint32_t* b, uint32_t* out, int n) {
    int i = 0;
    for (; i <= n - 16; i += 16) {
        auto x = _mm512_loadu_si512(a + i);
        auto y = _mm512_loadu_si512(b + i);
        _mm512_storeu_si512(out + i, _mm512_add_epi32(x, y));
    }
    if (i < n) {
        __mmask16 mask = (1u << (n - i)) - 1;
        auto x = _mm512_maskz_loadu_epi32(mask, a + i);
        auto y = _mm512_maskz_loadu_epi32(mask, b + i);
        _mm512_mask_storeu_epi32(out + i, mask, _mm512_add_epi32(x, y));
    }
}

#pragma GCC pop_options
```

此处尾部长度为 1 到 15，因此移位合法；完整 16 项已由主循环处理。loadu 仅免除对齐要求，AVX2 完整加载仍不能跨数组边界；AVX-512 示例则明确使用掩码访存。

## 2.12 原生 Linux 实测

Ryzen 7 9700X、Linux x86_64、GCC16.2.1，O2 加 `-mavx2`。两个 $2^{20}$ 长度 uint32 数组相加，初始化和逐元素校验在计时外；每个进程执行 100 次，编译器屏障保留每轮写入，9 次交错运行取中位数。普通循环已在目标函数反汇编中确认使用 ymm 向量加法。

| 实现 | 编译器循环 | 手写 AVX2 | 手写 AVX512F |
|---|---|---|---|
| 毫秒 | 8.538 | 8.205 | 7.852 |

本例三者差距不大；换数据规模、访存方式或 CPU 后重新测量。先检查自动向量化，再对确实有收益的热点保留手写版本。

## 2.13 编译警告与检查配置

提交版用题目指定标准和优化选项；本地排错单独编译，别拿检查版耗时估算能否通过。常用命令如下，所有源文件同时编译时使用一致的宏和检查选项。

```
g++ main.cpp -std=c++23 -O2 -Wall -Wextra -Wshadow -Wconversion -o main
g++ main.cpp -std=c++23 -O1 -g -fsanitize=address,undefined \
    -fno-sanitize-recover=all -fno-omit-frame-pointer -o debug
g++ main.cpp -std=c++23 -O1 -g -D_GLIBCXX_DEBUG -o debug_stl
```

| 选项 | 用途与易错点 |
|---|---|
| `-Wall -Wextra` | 常见可疑写法；Wall 并不包含所有警告。 |
| `-Wshadow` | 同名局部变量遮蔽外层变量。 |
| `-Wconversion` | 可能改变数值的隐式转换；先核对范围，别为消警告随意强转。 |
| `-Wformat` | printf/scanf 格式与参数不匹配；Wall 已包含常用检查。 |
| `-fsanitize=address` | 常见堆栈越界、释放后使用；不能指望检出所有未初始化读取或容器逻辑越界。 |
| `-fsanitize=undefined` | 有符号溢出、非法移位等已覆盖的 UB；不是全部 UB 的检测器。 |
| `-fno-sanitize-recover=all` | 检出问题立即停止，不让 UBSan 报错后继续跑出“答案”。 |
| `-D_GLIBCXX_ASSERTIONS` | libstdc++ 轻量前提检查，如 vector 下标；不跟踪全部迭代器失效。 |
| `-D_GLIBCXX_DEBUG` | 更强容器/迭代器检查；有额外开销，部分容器类型与布局改变。 |

_GLIBCXX_DEBUG 与普通模式不能随意跨编译单元传同一个 STL 容器。ASan 判断可访问内存，vector 的 size 与 capacity 是另一层约束；在已分配容量内访问 size 外元素也非法，但未启用容器注解的 ASan 可能漏掉，要用下标检查或 `at()`。无符号回绕是定义良好的行为，普通 UBSan 不会把它当成有符号溢出。检查只覆盖实际执行路径，仍要用边界数据和对拍。

下例只编译看警告：截断小数、遮蔽变量、printf 类型不匹配、未使用变量。不要执行存在格式类型错误的程序。

```
#include <bits/stdc++.h>
using namespace std;

// Deliberately suspicious: inspect warnings, do not run this example.
int main() {
    int value = 7;
    double fraction = 1.5;
    int truncated = fraction;
    for (int value = 0; value < 2; value++) {
        cout << value << '\n';
    }
    printf("%lld\n", truncated);
}
```

## 2.14 读懂第一条错误

保存下面故意写错的程序为 diagnostics.cpp。每次只执行对应编号；报错后先找第一条诊断和回溯中自己代码的位置，不追最后一次崩溃。

```cpp
#include <bits/stdc++.h>
using namespace std;

// Deliberately incorrect: run only to demonstrate the named checker.
int main(int argc, char** argv) {
    if (argc != 2) {
        return 2;
    }
    int mode = stoi(argv[1]);
    if (mode == 1) {
        volatile int x = INT_MAX;
        cout << x + 1 << '\n';
    } else if (mode == 2) {
        auto p = make_unique<int[]>(1);
        volatile int index = 1;
        cout << p[index] << '\n';
    } else if (mode == 3) {
        vector<int> a(1);
        a.reserve(8);
        cout << a[3] << '\n';
    } else if (mode == 4) {
        vector<int> a{1, 2};
        auto it = a.begin();
        a.erase(it);
        cout << *it << '\n';
    }
}
```

| 参数 | 编译时加入 | 典型报错关键词 |
|---|---|---|
| 1 | `-fsanitize=undefined` `-fno-sanitize-recover=all` | signed integer overflow |
| 2 | `-fsanitize=address` | heap-buffer-overflow |
| 3 | `-D_GLIBCXX_ASSERTIONS` | 下标小于 size 的断言失败 |
| 4 | `-D_GLIBCXX_DEBUG` | singular iterator |

例如 `g++ diagnostics.cpp -std=c++23 -O1 -g -D_GLIBCXX_DEBUG -o debug`，再执行 `./debug 4`。诊断文本随版本变化，重点看错误类型、访问大小与源代码行。

## 2.15 值、引用与调用边界

参数 T 按值，const T& 只读借用，T& 允许修改原对象。返回引用必须保证被引用对象仍存活，不能返回局部变量的引用。结构化绑定 auto [x,y] 默认复制，auto& [x,y] 引用原元素；只读遍历常用 const auto&。

## 2.16 std::move：允许移动，不是立即移动

std::move(x) 本身是转换表达式，使后续重载选择可以使用移动构造或移动赋值；它自己不搬运数据。对 const 对象使用 move 不会去掉 const，常见容器因此仍走复制。除有更强保证的特例外，标准库对象移动后有效但状态未指定：可销毁、重新赋值，或调用满足其前提的操作，不假定内容保持原样或必定为空。

```
vector<int> a{1, 2, 3};
vector<int> b = std::move(a);
a.clear();
a.push_back(7);
const vector<int> c{4, 5};
vector<int> d = std::move(c);
```

上例 b 得到原序列，d 从 const c 复制。调用接受值参数的卷积函数且之后不再需要输入时，传 std::move(a) 可以避免额外副本；函数参数为 const 引用时不会因此取得所有权。返回本地变量通常直接 return a;；多加 move 可能失去具名返回值优化机会。

## 2.17 vector、array、deque 与字符串

vector 连续存储，随机访问常数时间，尾部追加摊还常数时间。reserve 只改容量，不增加元素；resize 才改变 size。重分配使原指针、引用和迭代器失效；erase 使被删位置及之后失效。不要保存 vector 节点引用后再 push 新节点；需要跨扩容引用时保存下标并重新访问。

array 的长度为编译期常量；局部基本类型元素需用 {} 初始化。deque 支持两端操作且可下标访问，但整体不连续；修改后不要盲目保留迭代器。vector<bool> 的元素可能是代理对象，不等同于普通 bool 引用。

string 适合字符序列；substr(pos,len) 第二个参数是长度。find 未找到返回 npos，不能直接转 int 再判断。getline 读整行，与 cin>> 混用时处理遗留换行。字符串反复头部删除可能线性搬移；需要复杂序列操作见分册 rope 与序列树。

## 2.18 set、map 与哈希表

set/map 去重，multiset/multimap 允许等价键。比较器必须是严格弱序；不能用 less_equal 假装多重集合，也不直接用不传递的 epsilon 比较做排序或树比较。map 的下标访问会插入默认值，只查询用 find。

浮点键若含 NaN，普通小于比较也不再满足严格弱序：NaN 与不同有限数都可能互不小于，但这些有限数之间仍有大小关系。几何排序前应遵守有限输入约定，不能把 NaN 或 epsilon 当成普通相等键。

multiset 的 erase(key) 会删掉全部等价元素；只删一个时先 find，再 erase 该迭代器。树容器的 lower_bound 用成员函数，通用算法在非随机访问迭代器上可能线性走步。关联容器的键不可原地修改为破坏排序的值。

**比较器决定等价与边界** 等价指两个方向比较都为假，不一定是字段逐一相等。begin 指向首元素，end 是尾后位置，不能解引用；空集合两者相同。lower_bound(k) 找第一个“不排在 k 前面”的元素，upper_bound(k) 找第一个“排在 k 后面”的元素；使用降序比较器时，不再是数值上的大于等于/大于。

```
multiset<int, greater<int>> s{7, 5, 5, 2};
auto l = s.lower_bound(5); // 指向第一个5
auto r = s.upper_bound(5); // 指向2
s.erase(l);              // 只删除一个5
s.erase(5);              // 删除剩余全部5
```

结构体可写独立比较器；以下按 x 升序、x 相同按 y 降序。同一比较器可用于 set 去重或 multiset 保留重复值。查找返回 end 时不能 erase 该迭代器；equal_range 返回等价键的半开区间，clear 清空，size/count 分别给出总数/等价键数。

```
struct Node
{
    int x, y;
};
struct Cmp
{
    bool operator()(const Node &a, const Node &b) const
    {
        if (a.x != b.x)
            return a.x < b.x;
        return a.y > b.y;
    }
};
multiset<Node, Cmp> nodes;
```

unordered_map 的普通查改平均常数时间，最坏可线性；rehash 会使迭代器失效。需要容量控制时看其 reserve/max_load_factor 接口，不把这些名字照搬到 PBDS。GNU 的 gp/cc 哈希表、选型和范围见本册末的分册索引。

## 2.19 queue、stack 与优先队列

queue 先入先出，stack 后入先出；取 front/top 和 pop 前检查非空。标准 priority_queue 默认大根堆，pop 不返回值，先复制 top 再 pop。它没有按句柄修改和合并接口。

```
priority_queue<int, vector<int>, greater<int>> q;
q.push(7);
q.push(2);
int x = q.top();
q.pop();
```

比较器 comp(a,b) 为真表示 a 的优先级低于 b；相同优先级不保证先进先出。需要按优先级再按编号取最小时，pair 的字典序已足够：

```
using Item = pair<int, int>; // 优先级、编号
priority_queue<Item, vector<Item>, greater<Item>> q2;
q2.push({3, 8});
q2.push({3, 2});
auto [key, id] = q2.top(); // 3、2
q2.pop();
```

也可提供独立比较器；无需同时重载多个关系运算符。比较器若读取外部数组，入堆后不能直接修改数组来假装降键：堆不会自动重排。应存入键值快照并在取出时过滤过期项（见最短路），或使用支持修改的堆。

需要修改已入堆的键、删除指定元素或合并时，使用数据结构册的 PBDS 配对堆；不要混用两种 priority_queue 的接口或句柄保证。

## 2.20 排序、查找与区间算法

算法区间通常为半开区间。sort 要求严格弱序，stable_sort 保留等价元素的原顺序；nth_element 只保证第 k 位及两侧分区，平均线性，不等于完整排序。lower/upper_bound 要求对应有序或分区前提，并使用一致比较规则。

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
auto it = lower_bound(a.begin(), a.end(), 7);
if (it != a.end()) {
    cout << *it << '\n';
}
```

unique 只合并相邻等价值，返回逻辑终点；它不改容器 size。remove/remove_if 同理，配合 erase；C++20 可用适用容器的 erase/erase_if。reverse、rotate、merge、copy 的目标区间必须有效，不能把未扩容的 vector.begin() 当作足够大的输出区间。

## 2.21 pair、tuple、optional 与回调

pair/tuple 默认按字典序比较，适合把值与唯一编号组合。optional 区分无解与合法零值，先判断再解引用。`std::function` 可保存不同可调用对象，但有类型擦除开销；泛型 lambda 或模板回调更容易内联。递归 lambda 捕获的引用不能活得比被引用对象更久。

`tie(a,b)` 创建引用元组，保存后仍引用原变量，不是值快照。次小生成树用它比较权值与编号；模整数扩展欧几里得用右侧的值 pair 先保存旧值，再赋给左侧引用。不能把 `tie(a,b)=tie(b,a)` 当作可靠的交换：右侧也在引用会被改写的变量。

```
long long a = 17, b = 5;
long long q = a / b;
tie(a, b) = pair{b, a - q * b};
assert(a == 5 && b == 2);
```

普通数值变量要保存快照可用 `pair{a,b}` 或 `make_tuple(a,b)`，交换则直接 swap。

## 2.22 数值累加与前缀和

accumulate 的初值决定累加类型，64 位求和用 `0LL`；把返回结果装进 long long 不会改变中间累加类型。乘积也须先提升，例如 `1LL*a*b`，但仍应保证宽类型可表示结果。

partial_sum 的累加器类型取自输入迭代器的值类型，不由输出容器决定。SAM 的长度桶前缀计数用 int，前提是总状态数可表示；更大的前缀和应先将输入转为 long long，或用显式的宽类型循环。只把输出改为 vector<long long> 不能避免 int 累加溢出。允许输出起点等于输入起点：

```
vector<int> a{1000000000, 1000000000, 1000000000};
vector<long long> prefix(a.begin(), a.end());
partial_sum(prefix.begin(), prefix.end(), prefix.begin());
assert(prefix.back() == 3000000000LL);
```

空区间不写输出；非空时输出区间必须足够大。iota 适合连续编号。gcd/lcm 仍有可表示范围前提，最小负数的绝对值应先转宽或转无符号幅值。浮点整数精确范围和 long double 有效位数用 numeric_limits 查询；几何数值条件另看几何册。

## 2.23 complex 与 FFT

FFT 使用 `complex<long double>`；`C(re,im)` 分别设置实部、虚部，real()/imag() 读取分量，conj(z) 取共轭。abs(z) 是模长，norm(z) 是模长的平方；尤其不要与本库 RealPlane::norm 的长度含义混淆。逆变换使用共轭单位根，并按模板约定除以长度。

```
using C = complex<long double>;
C z(3, 4);
assert(z.real() == 3 && z.imag() == 4);
assert(conj(z) == C(3, -4));
assert(abs(z) == 5 && norm(z) == 25);
```

复数乘法仍是浮点运算，norm 也可能溢出，不能因 API 名称而放宽 FFT 的输入范围与舍入误差界。精确整数卷积是否适用浮点版本，应看数学册说明。

## 2.24 C++20 位操作与零边界

bit_width、popcount、countl_zero、countr_zero 接受无符号整数类型。bit_width(x) 表示 x 所需的二进制位数，零返回 0；popcount 返回 1 的个数。两个 count-zero 函数对零都返回该类型的位宽，不是 0。不要把 countl_zero 的结果直接当作最高位下标。

```
static_assert(bit_width(0u) == 0);
static_assert(bit_width(8u) == 4);
static_assert(popcount(10u) == 2);
static_assert(countr_zero(0u) == numeric_limits<unsigned>::digits);
```

划分树、WaveletMatrix 用 `bit_width((unsigned)max(1,n)-1)` 这类表达式计算层数；其中 n 分别取数组长度与去重后的值域大小。先保护零再减一，避免无符号回绕；各模板是否另加一层按接口说明。GCC 的 `__builtin_ctz(0)` 与 `__builtin_clz(0)` 均不能调用，处理 64 位输入还须选对应 ll 版本。

`1ULL<<k` 须满足 k 非负且小于 unsigned long long 的位宽。WaveletMatrix 按 64 位块存储，用 `k=i%64` 得到 0 到 63；k=0 时 `(1ULL<<k)-1` 为零掩码，不能用移位 64 构造整块掩码。bitset 的大小固定，批量位运算不是与位数无关的常数操作。

## 2.25 计时与卡时

steady_clock 单调，适合测经过时间；system_clock 是日历时间，可能被校时。high_resolution_clock 可能只是前两者之一的别名，不能据名字假定单调。duration_cast<milliseconds> 截掉不足 1 毫秒的部分；duration<double> 得到以秒为单位的浮点数。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    using Clock = chrono::steady_clock;
    auto start = Clock::now();
    auto deadline = start + chrono::milliseconds(20);
    mt19937_64 rng(start.time_since_epoch().count());
    uint64_t checksum = 0;
    do {
        for (int i = 0; i < 1024; i++) {
            checksum ^= rng();
        }
    } while (Clock::now() < deadline);
    auto elapsed = Clock::now() - start;
    auto ms = chrono::duration_cast<chrono::milliseconds>(elapsed).count();
    double seconds = chrono::duration<double>(elapsed).count();
    cout << checksum << '\n';
    cerr << ms << " ms, " << seconds << " s\n";
}
```

上例每 1024 次操作检查一次时间，减少读钟开销；卡时可能超出一个批次，还要为输出留余量。时间种子可改变每次运行的序列；对拍改成固定种子，并保存失败输入。随机搜索要随时保存合法的当前最优解，不能等超时后才构造答案。

## 2.26 随机引擎、分布与抽样

mt19937 产生 32 位值，mt19937_64 产生 64 位值。不要用 rng()%n 代替均匀分布：除非区间大小整除引擎值域大小，否则有偏差。rand() 的值域和质量依实现；random_shuffle 已在 C++17 移除，改用 shuffle。

| 调用 | 结果与条件 |
|---|---|
| uniform_int_distribution<long long>(l,r) | 整数闭区间 $[l,r]$，要求 $l \le r$。 |
| uniform_real_distribution<double>(l,r) | 目标区间 $[l,r)$；用有限小范围且 $l < r$，注意浮点舍入。 |
| bernoulli_distribution(p) | 概率 $p$ 为 true，$0 \le p \le 1$。 |
| normal_distribution<double>(mu,sigma) | 正态分布；标准差 $\sigma > 0$，结果不限区间。 |
| discrete_distribution<int>{1,2,7} | 返回下标 0、1、2，概率比 1:2:7；权非负且总和正。 |
| shuffle(first,last,rng) | 原地打乱，保留元素和重复次数。 |
| sample(first,last,out,k,rng) | 无放回抽取至多 $k$ 个位置；值不一定互异。 |

分布对象可能缓存结果（例如正态分布）。重设引擎种子后，要复现整条调用序列，还须对已用分布调用 reset() 或重新构造。同一工具链中固定种子与调用顺序可复现；不同标准库的分布和洗牌结果不保证逐项相同。

## 2.27 对拍用造数据函数

以下代码保存为 random_tools.hpp；点编号为 $[0, n)$，长度非负。互异值先生成全集再抽取，适合可枚举的小值域；随机字符串的字母表非空。树采用逐点连接旧点再随机重标号，**并非所有标号树等概率**。简单无向图先枚举全部边再抽取，适合小数据，不保证连通。对拍还应专门构造链、星、完全图、空图、重复权等边界，别只依赖一种随机分布。

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

mt19937_64 rng(712367821);

long long random_int(long long l, long long r) {
    return uniform_int_distribution<long long>(l, r)(rng);
}

double random_real(double l, double r) {
    return uniform_real_distribution<double>(l, r)(rng);
}

vector<int> permutation(int n) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

// Choose k distinct values from [0, n), using O(n) space.
vector<int> distinct_values(int n, int k) {
    assert(0 <= k && k <= n);
    auto p = permutation(n);
    p.resize(k);
    return p;
}

string random_string(int n, const string& alphabet) {
    assert(!alphabet.empty());
    string s(n, ' ');
    for (char& c : s) {
        c = alphabet[random_int(0, (int)alphabet.size() - 1)];
    }
    return s;
}

// Random recursive tree, then relabel; not uniform over all trees.
vector<pair<int, int>> random_tree(int n) {
    auto p = permutation(n);
    vector<pair<int, int>> edges;
    for (int i = 1; i < n; i++) {
        edges.emplace_back(p[i], p[random_int(0, i - 1)]);
    }
    shuffle(edges.begin(), edges.end(), rng);
    return edges;
}
```

```cpp
// Small undirected simple graphs; O(n^2) time and space.
vector<pair<int, int>> random_graph(int n, int m) {
    assert(0 <= m && m <= 1LL * n * (n - 1) / 2);
    vector<pair<int, int>> edges;
    for (int u = 0; u < n; u++) {
        for (int v = u + 1; v < n; v++) {
            edges.emplace_back(u, v);
        }
    }
    shuffle(edges.begin(), edges.end(), rng);
    edges.resize(m);
    return edges;
}
```

## 2.28 生成器入口

下面入口与 random_tools.hpp 放在同一目录，编译后以命令行参数传种子。演示输出先打印各类随机值，最后打印图；对拍时只保留题目需要的输出格式。

```cpp
#include "random_tools.hpp"

int main(int argc, char** argv) {
    if (argc > 1) {
        rng.seed(stoull(argv[1]));
    }
    bernoulli_distribution coin(0.3);
    normal_distribution<double> normal(0.0, 1.0);
    discrete_distribution<int> weighted{1, 2, 7};
    cout << coin(rng) << ' ' << normal(rng) << ' ' << weighted(rng) << '\n';
    auto p = permutation(10);
    vector<int> chosen;
    sample(p.begin(), p.end(), back_inserter(chosen), 3, rng);
    for (int x : chosen) {
        cout << x << ' ';
    }
    cout << '\n';
    cout << random_int(-10, 10) << ' ' << random_real(0, 1) << '\n';
    cout << random_string(12, "abc") << '\n';
    auto edges = random_graph(6, 8);
    cout << 6 << ' ' << edges.size() << '\n';
    for (auto [u, v] : edges) {
        cout << u << ' ' << v << '\n';
    }
}
```

## 2.29 整数键防卡哈希

整数键可用时间种子配合 splitmix64 混合，降低针对固定哈希构造冲突的风险。它不保证最坏时间，也不适合密码用途。种子一经初始化，在容器存活期间保持不变；不要每次调用哈希时重新取时间。PBDS 没有与 unordered_map 相同的 reserve 接口。

```cpp
#include <bits/stdc++.h>
#include <cassert>
#include <ext/pb_ds/assoc_container.hpp>
using namespace std;

struct Hash {
    static uint64_t mix(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};

int main() {
    unordered_map<long long, int, Hash> a;
    __gnu_pbds::gp_hash_table<long long, int, Hash> b;
    a.reserve(1000);
    for (long long x : {-1LL, 0LL, LLONG_MIN, LLONG_MAX}) {
        a[x]++;
        b[x]++;
        assert(a.at(x) == b.find(x)->second);
    }
    cout << a.size() << ' ' << b.size() << '\n';
}
```

## 2.30 GNU 扩展与查询范围

PBDS 有序树、gp/cc 哈希表、配对堆以及 `__gnu_cxx::rope` 在所属知识点详细介绍，本册提供集中页码跳转。rope 不叫 std::rope；复制版本的内存成本和已知失败用法不因它是库容器而消失。

## 2.31 Boost 任意精度整数

`boost::multiprecision::cpp_int` 是有符号任意精度整数，须显式包含 `boost/multiprecision/cpp_int.hpp`；它不属于 ISO C++ 标准库或 GNU 标准头文件，赛场必须提供 Boost。可使用十进制流输入输出，整数除法仍是截断除法，不会自动变成有理数，除数不能为零。Bareiss 消元依靠数学上的整除性质保存精确整数，矩阵树用它计算不取模答案；目录末有对应分册跳转。数值越大，运算和存储越贵，不应把任意精度操作视为机器字常数时间。Boost 使用表达式模板，需跨语句保存计算结果时写明确的 `cpp_int` 类型，避免用 `auto` 保存依赖随后修改变量或临时量的表达式。

## 2.32 ___int128 的十进制读写

有符号范围 $[-2^{127}, 2^{127}-1]$，无符号范围 $[0, 2^{128}-1]$。先转换再乘，如 `i128(a)*b`；有符号溢出仍是 UB。最小负数不能直接取负，也不能先读到 long long 再转换。下面函数按字符串解析，拒绝非法或超范围输入；失败时保留原变量。输出先取无符号幅值，正确处理最小负数与 0。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i128 = __int128;
using u128 = unsigned __int128;

// Invalid or out-of-range token: return false, leave x unchanged.
bool parse128(const string& s, i128& x) {
    if (s.empty()) {
        return false;
    }
    bool neg = s[0] == '-';
    size_t pos = neg || s[0] == '+';
    if (pos == s.size()) {
        return false;
    }
    u128 limit = (u128(1) << 127) - !neg;
    u128 value = 0;
    for (; pos < s.size(); pos++) {
        if (s[pos] < '0' || s[pos] > '9') {
            return false;
        }
        unsigned digit = s[pos] - '0';
        if (value > (limit - digit) / 10) {
            return false;
        }
        value = value * 10 + digit;
    }
    x = neg && value ? -i128(value - 1) - 1 : i128(value);
    return true;
}

string str128(i128 x) {
    bool neg = x < 0;
    u128 value = neg ? u128(0) - u128(x) : u128(x);
    string s;
    do {
        s.push_back('0' + value % 10);
        value /= 10;
    } while (value);
    if (neg) {
        s.push_back('-');
    }
    reverse(s.begin(), s.end());
    return s;
}
```

## 2.33 strict 与 GNU 模式的差异

前页函数后接以下入口即可读到 EOF；普通 cin/cout 没有直接的 ___int128 数值重载。

```
int main() {
    string s;
    while (cin >> s) {
        i128 x = 0;
        if (parse128(s, x)) {
            cout << str128(x) << '\n';
        } else {
            cout << "invalid\n";
        }
    }
}
```

GCC 在支持 128 位整数的目标上，-std=c++20 也接受 ___int128 扩展，但它不是 ISO 标准类型。

加入 -pedantic-errors 会拒绝此扩展写法。-std=gnu++20 关闭 strict 模式，部分版本的标准库还会改变对扩展类型的识别，不能因为能声明变量就假定所有模板都接受它。

```
#include <iostream>
#include <limits>
#include <type_traits>
using namespace std;

int main() {
    cout << __cplusplus << '\n';
#ifdef __STRICT_ANSI__
    cout << 1 << '\n';
#else
    cout << 0 << '\n';
#endif
    cout << sizeof(__int128) << '\n';
    cout << is_integral_v<__int128> << '\n';
    cout << numeric_limits<__int128>::digits << '\n';
}
```

本机 GNU GCC16、libstdc++ 实测：以上探针在两种模式分别输出 202002 / 1 / 16 / 1 / 127 与 202002 / 0 / 16 / 1 / 127。本版本两种模式的 is_integral_v<__int128> 均为 true；区别是 strict 宏。标准库约束与可用接口还取决于版本。热身时用比赛的真实编译命令跑探针；不要用该 trait 在 strict 模式下否定编译器的算术支持。GNU 头文件、PBDS 和 bitset 扩展同样不是 ISO 接口；算法分册页码见本册末索引。

## 2.34 GNU 位函数与 bitset 遍历

| 写法 | 结果或条件 |
|---|---|
| `__builtin_popcountll(x)` | unsigned long long 中 1 的个数，允许 0。 |
| `__builtin_ctzll(x)` | 从低位数连续 0；$x \neq 0$。 |
| `__builtin_clzll(x)` | 从高位数连续 0；$x \neq 0$，64 位时最高位下标为 63 减该值。 |
| `x &= x-1` | 对无符号非零数删去最低位的 1。 |
| `b._Find_first()` | 最低置位位置，无置位返回 `b.size()`。 |
| `b._Find_next(i)` | 严格大于 $i$ 的下一置位，无则返回 size。 |

无 ll 的版本对应 unsigned int；64 位值误传会被截断。bitset 两个查找函数是 libstdc++ 扩展，可跨多个机器字扫描，并非总是常数时间。

以下例子同时覆盖第 0、64、129 位以及空集合；遍历条件必须排除 size 哨兵。

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

vector<int> positions(uint64_t x) {
    vector<int> p;
    while (x) {
        p.push_back(__builtin_ctzll(x));
        x &= x - 1;
    }
    return p;
}

int main() {
    uint64_t x = (1ULL << 63) | 5;
    assert(__builtin_popcountll(x) == 3);
    assert(63 - __builtin_clzll(x) == 63);
    assert(positions(x) == vector<int>({0, 2, 63}));
    assert(positions(0).empty());
    bitset<130> b;
    b.set(0);
    b.set(64);
    b.set(129);
    for (size_t i = b._Find_first(); i < b.size(); i = b._Find_next(i)) {
        cout << i << '\n';
    }
    b.reset();
    assert(b._Find_first() == b.size());
    return 0;
}
```

# 第三章　Shell 脚本与常用命令

## 3.1　编译并运行

保存为 compile.sh，运行 `bash compile.sh main`；默认调试编译后运行。加 `-O2` 使用优化配置；`-o solution` 指定输出且只编译。输入输出可在调用后接 `< input.txt > actual.txt`。

```
#!/usr/bin/env bash
set -euo pipefail
file=
out=
run=1
flags=(-Wall -Wextra -Wno-sign-compare -std=c++23)
mode=(-O1 -g -fsanitize=address,undefined -fno-sanitize-recover=all)
while (($#)); do
    case "$1" in
        -O2)
            mode=(-O2)
            ;;
        -o)
            [[ $# -ge 2 && $2 != -* ]] || exit 2
            out=$2
            run=0
            shift
            ;;
        -*)
            echo "unknown option: $1" >&2
            exit 2
            ;;
        *)
            [[ -z $file ]] || exit 2
            file=$1
            ;;
    esac
    shift
done
[[ -n $file ]] || exit 2
file="${file%.cpp}.cpp"
[[ -f $file ]] || exit 2
out=${out:-${file%.cpp}}
[[ $out == /* ]] || out="./$out"
[[ ! -L $out && ! $file -ef $out ]] || exit 2
[[ ! -e $out || ( -f $out && -x $out ) ]] || exit 2
g++ "$file" -o "$out" "${flags[@]}" "${mode[@]}"
echo "compiled: $out" >&2
if ((run)); then
    "$out"
fi
```

只支持单源文件，文件名不以减号开头。路径含空格时加引号。拒绝覆盖源码、普通文本、目录和链接；已有普通可执行文件可覆盖。编译失败不运行旧文件；程序运行失败保留非零退出状态。

## 3.2 对拍与保存反例

当前目录准备可执行的 gen、brute、main：生成器接收整数种子，暴力和待测从标准输入读取。执行 `bash stress.sh 1000`。

```
#!/usr/bin/env bash
set -euo pipefail
count=${1:-1000}
[[ $count =~ ^[1-9][0-9]*$ ]] || exit 2
dir=$(mktemp -d ./stress.XXXXXX)
echo "saved in $dir" >&2
trap 'echo "stopped at seed $seed; files: $dir" >&2' ERR
for ((seed=1; seed<=count; seed++)); do
    echo "$seed" > "$dir/seed"
    for name in input expected actual diff gen.err brute.err main.err; do
        : > "$dir/$name"
    done
    timeout -k 1s 2s ./gen "$seed" > "$dir/input" 2> "$dir/gen.err"
    timeout -k 1s 2s ./brute < "$dir/input" > "$dir/expected" 2> "$dir/brute.err"
    timeout -k 1s 2s ./main < "$dir/input" > "$dir/actual" 2> "$dir/main.err"
    diff -u "$dir/expected" "$dir/actual" > "$dir/diff"
done
echo "passed $count seeds; last case: $dir"
```

每个程序分别限时 2 秒，TERM 后 1 秒仍未退出则 KILL。timeout 返回 124 通常为超时，137 可能为 KILL；检查对应 err 文件。diff 返回 1 表示答案不同，其他错误也停下。浮点容差题、输出方案题应换为专用 checker。每次创建独立目录，每轮清空输出槽；失败保存首个反例，成功保留末组。用保存的 input 直接重跑，不能只凭种子猜测输入。

## 3.3 完整造数与暴力示例

示例任务为非空最大子段和。保存下列生成器为 gen.py，暴力为 brute.py；待测 main 使用自己的算法。复制为 gen、brute 后执行 `chmod +x gen brute`，用前页脚本对拍。

```python
#!/usr/bin/env python3
import random
import sys

rng = random.Random(int(sys.argv[1]))
n = rng.randint(1, 12)
a = [rng.randint(-20, 20) for _ in range(n)]
print(n)
print(*a)
```

```python
#!/usr/bin/env python3
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
n = data[0]
a = data[1:]
assert len(a) == n
# Nonempty maximum subarray sum.
print(max(sum(a[l:r]) for l in range(n) for r in range(l + 1, n + 1)))
```

| 写法 | 用途与坑点 |
|---|---|
| `"$1"`、`$#`、shift | 首参数、参数个数、移除首参数；路径变量必须双引号。 |
| `flags=(...)` | 数组；`"${flags[@]}"` 逐项传参，不能把选项全塞入一个字符串。 |
| `[[ ... ]]`、`(( ... ))` | 字符串/文件判断、算术条件。算术结果 0 的退出状态是 1。 |
| `<`、`>`、`2>` | 输入、覆盖输出、错误输出；`>>` 追加，别把同一文件同时当输入输出。 |
| `set -euo pipefail` | 普通命令失败、未设置变量、管道失败尽早停止；if 条件和 &&/\|\| 链等有例外。 |
| `head -n 20 file` | 看前 20 行；`wc -l file` 数换行，`sort -n file` 数值排序。 |
| `diff -u a b` | 对比内容；`cat run.err` 检查错误。编译成功不等于运行成功。 |

# 第四章　赛场环境与调试

## 4.1　开赛与热身检查清单

选手机是编辑、编译、对拍的机器；评测机实际运行提交程序。选手机的 CPU、版本和限制不能代替评测配置。先读比赛提供的语言列表与完整编译命令；热身时用允许的测试方式提交短探针，记录实际结果。如果系统不展示程序输出，使用提供的自定义测试或向工作人员确认。

| 检查项 | 需要确认或完成的动作 |
| --- | --- |
| 编译器与标准 | 记录评测命令、GCC 版本、语言档位、O2、宏定义与栈设置；程序可输出 `__VERSION__` 和 `__cplusplus`，但不能由这两个值还原全部编译选项。 |
| 头文件与扩展 | 编译实际使用的 `__int128`、PBDS、Boost 探针；只包含头文件而不用接口，检查不充分。 |
| CPU 与指令集 | 本机用 lscpu；评测探针用 CPU 内建检测 AVX2/AVX512F 等。以基础目标编译探针，先检测再运行目标指令；不要用`-march=native` 猜评测 CPU。 |
| 栈与内存 | 核对题目内存、栈限制及 getrlimit 结果；栈软上限与硬上限分别记录。DFS 保留递归，本地栈过小就调整测试限制。 |
| 大致速度 | 同一份有可核验输出的 O2 基准分别跑本机与评测机；记录数据规模，不用一次微基准推算所有算法。 |
| 编辑器 | 保存 UTF-8；设置 4 空格缩进、显示行号；配置第三章编译脚本、输入重定向和运行目录，亲自编译运行样例。 |
| 常用操作 | 练熟保存、跳转行号、查找/替换、整行注释、缩进、撤销及终端切换；以现场编辑器实际绑定为准。 |
| 提交链路 | 提交一次样例程序，确认文件、语言、输入输出格式及查看编译错误的位置。交互题另测刷新与失败退出。 |

本地终端检查；这些结果只属于当前选手机：

```
g++ --version
python3 --version
lscpu
ulimit -Ss
ulimit -Hs
g++ environment.cpp -std=c++23 -O2 -o environment
./environment
```

Vim 可在命令模式输入 `:set number expandtab shiftwidth=4 tabstop=4`；`:w` 保存，`:42` 跳转行，`/word` 查找，n 到下一处，u 撤销，Ctrl-R 重做。使用 VS Code 时可练熟 Ctrl-S 保存、Ctrl-G 跳行、Ctrl-F 查找、Ctrl-H 替换、Ctrl-/注释；现场先确认键位未被桌面拦截。

## 4.2 短探针与速度校验

以下保存为 environment.cpp，面向 Linux x86/GCC；编译成功并输出 9 同时验证 PBDS 可用。getrlimit 单位为字节，RLIM_INFINITY 表示无限制；这仍须与比赛公布的限制一起解释。速度循环保留可观察校验值，避免无用循环被删除。

```cpp
#include <bits/stdc++.h>
#include <sys/resource.h>
#include <ext/pb_ds/assoc_container.hpp>
using namespace std;

int main() {
    cout << "compiler " << __VERSION__ << '\n';
    cout << "standard " << __cplusplus << '\n';
    cout << "int128 " << sizeof(__int128) << '\n';
    __gnu_pbds::gp_hash_table<int, int> table;
    table[7] = 9;
    cout << "pbds " << table[7] << '\n';
    rlimit limit;
    if (getrlimit(RLIMIT_STACK, &limit) != 0) {
        return 1;
    }
    cout << "stack soft/hard " << limit.rlim_cur << ' ';
    cout << limit.rlim_max << " bytes; infinity=" << RLIM_INFINITY << '\n';
    cout << "avx2 " << bool(__builtin_cpu_supports("avx2")) << '\n';
    cout << "avx512f " << bool(__builtin_cpu_supports("avx512f")) << '\n';
    uint64_t x = 1;
    auto start = chrono::steady_clock::now();
    for (int i = 0; i < 10000000; i++) {
        x ^= x << 13;
        x ^= x >> 7;
        x ^= x << 17;
    }
    double seconds = chrono::duration<double>(chrono::steady_clock::now() - start).count();
    cout << "checksum " << x << " seconds " << seconds << '\n';
}
```

CPU 检测只报告运行环境允许的特性；缺少 AVX-512 时不执行对应函数。评测系统禁用系统接口时遵守现场提供的环境说明，不把探针失败当作算法失败。若要使用 Boost，再编译数学册 cpp_int 的实际例子；它不由 PBDS 成功推出。

## 4.3 提交前核对

逐题确认下标、区间开闭、空输入、重边、自环、数值范围和模板前提。整数先转宽再乘；0 可能是合法答案。调试信息放 stderr，正式提交关闭 LOCAL。保留失败输入，先查样例、边界与小数据参考，再跑最大数据；检查版和 O2 版分别通过。

## 4.4 GDB、assert 与 LOCAL

用 -O0 -g 保留容易对应的源码位置；O2 可能内联、重排或移除变量。assert 条件必须没有需要保留的副作用；定义 NDEBUG 会删除断言，不能在断言内完成输入、赋值或必要调用。

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

int square(int x) {
    assert(0 <= x && x <= 10000);
#ifdef LOCAL
    cerr << "x=" << x << '\n';
#endif
    int result = x * x;
    return result;
}

int main() {
    int x;
    if (!(cin >> x)) {
        return 1;
    }
    int answer = square(x);
    cout << answer << '\n';
}
```

保存为 debug_local.cpp；输入 7 应输出 49，LOCAL 开启时 stderr 额外输出 x=7。

```
g++ debug_local.cpp -std=c++23 -O0 -g -DLOCAL -o debug
printf '7\n' > input.txt
gdb ./debug
```

| GDB 命令 | 用途 |
|---|---|
| break square / b file.cpp:20 | 在函数或源码行设断点。 |
| run < input.txt | 重新启动并从文件读输入。 |
| next / step | 单步越过函数 / 单步进入函数；简称 n/s。 |
| print x / display x | 立即查看 / 每次停止时显示变量。 |
| backtrace / frame 1 | 查看调用栈 / 切换到指定栈帧；简称 bt/f。 |
| info locals / list | 当前局部变量 / 附近源码。 |
| until 11 / finish | 运行到当前帧第 11 行 / 运行到当前函数返回。 |
| continue / quit | 继续到下个断点或退出 / 退出调试器。 |
| watch x / delete 1 | 在值改变时停下 / 删除编号 1 的断点。 |

本例可依次输入 b square、run < input.txt、p x、bt、n、until 11、p result、finish、c。预期看到 x=7、result=49 和 main 调用帧。输入负数会在 assert 处停止；看第一处异常与调用栈。

## 4.5 测时间、峰值内存与限制

Bash 的 time 报告 real（墙上时间）、user（用户态 CPU 时间）、sys（内核态 CPU 时间）；等待输入和调度会增加 real，多线程的累计 CPU 时间也可能大于 real。GNU time 的%e 是秒，%M 是最大驻留集 KiB，诊断输出走 stderr。

```
time ./memory
/usr/bin/time -v ./memory
/usr/bin/time -f '%es %MKB' ./memory
(ulimit -Sv 262144 && ulimit -Ss 65536 && ./memory)
timeout -k 1s 2s ./main < input.txt > output.txt
```

上例在子 shell 中限制 256MiB 虚拟地址空间、64MiB 栈，只调整软上限；继承的硬上限不足时设置会失败。用 `&&` 连接设置与运行，设置失败就停止。`ulimit -v` 约束虚拟地址空间，并不等同于 OJ 按 RSS 统计的内存；不要用它限制 ASan，ASan 需要很大的虚拟地址空间。`timeout` 用墙上时间，2 秒先发 TERM，再过 1 秒发 KILL；124 通常表示超时，137 可能表示 KILL。递归 DFS 不用改写为显式栈；根据题目内存预算配置本地栈，并保留最坏深度测试。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    clock_t start = clock();
    vector<int> a(1000000);
    iota(a.begin(), a.end(), 0);
    cout << accumulate(a.begin(), a.end(), 0LL) << '\n';
    ifstream status("/proc/self/status");
    for (string line; getline(status, line); ) {
        if (line.starts_with("VmPeak:") || line.starts_with("VmHWM:")) {
            cerr << line << '\n';
        }
    }
    clock_t finish = clock();
    if (start != clock_t(-1) && finish != clock_t(-1)) {
        cerr << "CPU seconds " << double(finish - start) / CLOCKS_PER_SEC << '\n';
    }
}
```

`clock()` 返回进程消耗的 CPU 时间刻度，除以 CLOCKS_PER_SEC 转秒；失败值为 `clock_t(-1)`。需要经过时间或卡时用第二章 steady_clock，不把 CPU 时间当墙上时间。`/proc/self/status` 里 VmPeak 是虚拟地址空间峰值，VmHWM 是驻留集高水位，输出 kB 按 KiB 解释。该接口的 RSS 统计可能近似；当前进程的数值不包含所有子进程，也不必与 OJ 统计口径一致。

| 对象 | 内存估算 |
|---|---|
| 连续数组 | `sizeof(T)*n`，先用 size_t 或更宽类型计算乘积，另计其他数组。 |
| vector | 元素缓冲按 capacity 而非 size 计算；扩容时新旧缓冲可能同时存在。二维 vector 还含每行对象与独立分配开销。 |
| map/set | 除键值外还有父/子指针、颜色、对齐和分配器开销；`sizeof(map)` 只测容器对象，不含节点。 |
| 递归栈 | 每层局部变量、返回地址、保存寄存器等乘最大深度；内联、优化与检查器都会影响帧大小，实际测最坏链。 |

## 4.6 交互题：刷新、协议与失败退出

示例：裁判藏一个 1 到 100 的整数。选手输出`? m`，裁判回复 1 表示答案不大于 m，否则回复 0；最多 7 次询问。输出`! x`报告答案。每次询问后必须刷新；关闭 cin.tie 后不能依赖下一次输入自动刷新。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int l = 1;
    int r = 100;
    while (l < r) {
        int m = (l + r) / 2;
        cout << "? " << m << endl;
        int reply;
        if (!(cin >> reply) || reply == -1) {
            return 1;
        }
        if (reply == 1) {
            r = m;
        } else {
            l = m + 1;
        }
    }
    cout << "! " << l << endl;
}
```

上例使用 endl 同时换行并刷新；也可写换行后 flush，C stdio 用 fflush(stdout)。收到 EOF 或题面规定的错误回复立即退出，不能继续输出询问。调试信息写 stderr，stdout 只留协议内容。

```
g++ interactive.cpp -std=c++23 -O2 -o interactive
timeout -k 1s 3s python3 interactor.py 42 ./interactive \
    2> interaction.log
cat interaction.log
```

下一页裁判由 Python 子进程管道连接选手，不用单向的`judge | solution`冒充双向通信。裁判每行记录双方内容，并检查询问次数、最终答案、退出码和多余输出。外层 timeout 约束整个进程组；如果双方互等，查看记录的最后一行，先检查谁没有刷新。本地裁判的协议应逐项对应题面：范围、询问预算、错误哨兵及最终输出格式。

## 4.7 最短本地裁判

保存为 interactor.py；第一个参数是秘密数，第二个是选手可执行文件。只适用于本页约定的单行协议，长输出、并发交互或多组数据按题意调整。

```
import subprocess
import sys

secret = int(sys.argv[1])
assert 1 <= secret <= 100
p = subprocess.Popen([sys.argv[2]], stdin=subprocess.PIPE,
                     stdout=subprocess.PIPE, text=True, bufsize=1)
try:
    queries = 0
    while True:
        line = p.stdout.readline()
        print('solution:', line.rstrip(), file=sys.stderr, flush=True)
        parts = line.split()
        assert len(parts) == 2, 'EOF or malformed output'
        kind, value = parts[0], int(parts[1])
        if kind == '!':
            assert value == secret, 'wrong answer'
            p.stdin.close()
            assert p.wait(timeout=1) == 0, 'nonzero exit'
            assert not p.stdout.read().strip(), 'extra output'
            break
        assert kind == '?' and 1 <= value <= 100
        queries += 1
        assert queries <= 7, 'too many queries'
        reply = int(secret <= value)
        print('judge:', reply, file=sys.stderr, flush=True)
        p.stdin.write(str(reply) + '\n')
        p.stdin.flush()
    print('AC', file=sys.stderr)
finally:
    if p.poll() is None:
        p.kill()
    p.wait()
```

裁判的 readline 会等待完整一行；必须使用前页 timeout 命令限制挂起。断言失败、数字解析失败和子程序非零退出均视为本地测试失败；不要用 `python -O` 运行裁判，否则断言会被删除。finally 在普通异常时回收子进程；外层超时直接终止进程组。

## 4.8 分册跳转索引

请将本册与其他 PDF 放在同一目录。下表页码是分册印刷页码；链接直接指向条目，部分浏览器内置 PDF 阅读器可能不支持跨文件跳转。


| 内容 | 所属分册及页码 |
|---|---|
| PBDS 配对堆：修改、合并与句柄 | 数据结构，第 17 页 |
| PBDS 有序树：排名与第 k 小 | 数据结构，第 110 页 |
| GNU rope：序列与版本共享 | 数据结构，第 108 页 |
| gp/cc 哈希表：整数键映射 | 数据结构，第 1 页 |
| 优先队列的最短路应用 | 图论，第 93 页 |
| vector 与下标：树状数组 | 数据结构，第 43 页 |
| 扩容与下标：可持久化数组 | 数据结构，第 179 页 |
| 比较次序：后缀数组 | 字符串，第 32 页 |
| 数值精度约定：浮点几何 | 计算几何，第 1 页 |
| 移动大数组：NTT 卷积 | 数学，第 122 页 |
| complex 的平方模与共轭：FFT | 数学，第 112 页 |
| 位宽与压位计数：小波矩阵 | 数据结构，第 103 页 |
| tie 与值比较：次小生成树 | 图论，第 142 页 |
| 精确整数行列式：Boost cpp_int | 数学，第 230 页 |
| 不取模的带权生成树计数 | 图论，第 341 页 |
| PBDS 句柄删除：双端优先队列 | 数据结构，第 18 页 |
| PBDS modify 降键：最短路与路径 | 数据结构，第 19 页 |
| move 数组：批量逆元与动态模数 | 数学，第 23 页 |

# 用途索引


accumulate, 20
array, 17
ASan, 15, 28
AVX-512, 14
AVX2, 12

bit-width, 20
bitset, 20, 27
Boost, 24
builtin, 27

chrono, 21
complex, 20
conj, 20
cpp-int, 24

deque, 17
diff, 29

EOF, 9

function, 19

GDB, 33
getline, 11
GNU 扩展, 26

heapq, 2

int128, 25
intrinsics, 12
istringstream, 11
itertools, 2

LOCAL, 33
lower-bound, 19

map, 17
multiset, 17

nth-element, 19
numeric, 20

O2, 5
O3, 5
Ofast, 5
optional, 19

pair, 19
partial-sum, 20
PBDS, 24
popcount, 20
pragma, 5
priority-queue, 18
Python, 1
Python 输入输出, 4

queue, 18

random, 21
reference, 17
reserve, 7
rope, 24

sample, 21
set, 17
shuffle, 21
SIMD, 12
SIMD 尾部, 13
SIMD 重排, 14
sort, 19
splitmix64, 24
stack, 18
std::move, 17
string, 17

time, 34
timeout, 29
tuple, 19

UBSan, 15, 28
ulimit, 34
unique, 19
unordered-map, 17

vector, 17

交互通信记录, 36
交互题, 35
低位 1, 27

内存布局, 7
列表别名, 4
刷新输出, 35
卡常, 7
卡时, 21
大整数, 1
对拍, 29

性能测试, 6
打表, 3
整数读入, 9
整数转字符串, 4
断点, 33
暴力, 30
最小负数, 25
模逆, 1
测内存, 34
浮点重排, 6

爆栈, 31
环境检查, 31
缓冲区, 8
编译脚本, 28

自动向量化, 12
警告, 15
计时, 21
记忆化, 2
语言标准, 26
越界, 15
输入输出优化, 8
运行时排错, 16
迭代器失效, 16

造数据, 3, 22
防卡哈希, 24
随机图, 22
随机树, 22